RAPPORT DE TP2 — SECURE MESSAGING

Introduction :

Dans ce mini-projet, nous avons construit une petite messagerie sécurisée : Alice écrit un message et Bob est le seul à pouvoir le lire. Nous voulions garantir trois choses : que personne d'autre ne puisse lire le message (confidentialité), que Bob s'aperçoive si le message a été modifié en route (intégrité) et qu'il soit sûr que c'est bien Alice qui l'a envoyé (authenticité). Comme RSA est lent et ne peut pas chiffrer de longs messages, nous avons choisi un chiffrement hybride : AES chiffre le message, et RSA sert uniquement à protéger la clé AES.

Comment ça marche :

Étape	Côté Alice (send)	Côté Bob (recv)
0 Les clés	Avec genkeys, chacun crée sa paire de clés RSA-2048 (alice_priv.pem, alice_pub.pem, etc.). Les clés publiques sont échangées.	Bob garde bob_priv.pem pour lui et récupère alice_pub.pem.
1 Clé de session	Alice tire au hasard une clé K de 32 octets, valable pour ce message seulement	Bob retrouve K en déchiffrant key_enc avec sa clé privée
2 Message	Elle chiffre message.txt avec AES-256-GCM et un nonce unique de 12 octets. On obtient le ciphertext et un tag de 16 octets.	Bob déchiffre avec AES-GCM. Si le tag ne correspond pas, le message est refusé
3 La clé K	Elle chiffre K avec la clé publique de Bob (RSA-OAEP) : cela donne key_enc.	
4 Signature	Elle signe le paquet final (key_enc‖nonce‖tag‖ciphertext) avec sa clé privée (RSA-PSS).	Bob vérifie d'abord la signature : OK ou KO. Si c'est KO, il s'arrête là.
5 Résultat	Le tout est écrit dans message.pkg (JSON + base64)	Bob obtient recovered.txt, identique au message d'origine

Nos choix et pourquoi :

Rôle	Choix	Pourquoi ?
Chiffrer le message	AES-256-GCM	Il chiffre et vérifie l'intégrité en même temps, et il est rapide.
Protéger la clé K	RSA-2048 OAEP (SHA-256)	OAEP ajoute de l'aléa et résiste mieux aux attaques que l'ancien PKCS#1 v1.5.
Signer	RSA-PSS + SHA-256	C'est la version moderne et recommandée de la signature RSA
Nonce et clé K	Valeurs aléatoires neuves à chaque envoi	Réutiliser un nonce avec la même clé casserait la sécurité de GCM : nous ne le faisons jamais
Hachage	SHA-256	Standard actuel, sans attaque pratique connue.

Le format de message.pkg (JSON, valeurs en base64) :

{ "version": 1, "key_enc": "<RSA-OAEP(K), 256 o>", "nonce": "<12 o>", "tag": "<16 o>", "ciphertext": "<n o>", "signature": "<RSA-PSS, 256 o>" }

Nous avons choisi de signer le paquet déjà chiffré (chiffrer puis signer). Ainsi, Bob peut rejeter un paquet falsifié avant même de le déchiffrer.

Ce qui manque encore, et comment l'améliorer :

Limites de notre prototype	Ce que nous ferions ensuite
Pas de PKI : nous faisons confiance aux clés publiques reçues, ce qui laisse la porte ouverte à une attaque de l'homme du milieu.	Utiliser des certificats X.509 ou comparer les empreintes des clés.
Pas de protection contre le rejeu : un ancien message.pkg reste accepté.	Ajouter une date, les identités et un compteur dans les données signées.
Les clés privées sont stockées en clair dans les fichiers PEM	Les protéger par mot de passe (scrypt/PBKDF2) ou par un coffre de clés
Pas de forward secrecy : si bob_priv.pem fuite, les anciens messages sont lisibles.	Passer à ECC (X25519 + HKDF, Ed25519) avec des clés éphémères.
Le fichier est lu en entier en mémoire, et il n'y a pas de révocation de clés	Traiter les gros fichiers par blocs et gérer la rotation/révocation des clés.

Conclusion :

Notre prototype répond bien au cahier des charges : le message est confidentiel (AES-GCM + RSA-OAEP), protégé contre toute modification (tag GCM + signature) et authentifié (RSA-PSS). C’est une bonne base pour un TP, mais il faudrait une vraie PKI et une protection contre le rejeu avant un usage réel.